深入AI Agent内核: Google gemini-cli 源码深度解构

原创 腾讯程序员 腾讯技术工程 2025年11月24日 01:38 广东 714人

作者：wenconglin

随着大语言模型与开发工具链的深度融合，命令行终端正被重塑为开发者的AI协作界面。本文以 Google gemini-cli 为范本，通过源码解构，系统性分析其 Agent 内核、ReAct 工作流、工具调用与上下文管理等核心模块的实现原理。为希望构建终端 Agent 的开发者，提供工程实现的系统化参考。

一、引言

命令行终端是开发者的核心工作区，也是许多开发者完成工作的首选工具。当前，大语言模型正在重塑命令行终端的能力，使终端不仅能完成命令执行，还能让开发者与智能体共同完成命令的协作、创造。

近期，Google开源了其官方实现的命令行AI工作流工具——**gemini-cli**（**https://github.com/google-gemini/gemini-cli**），旨在将 AI 能力深度融入开发者的日常。它能分析庞大的代码库、自动化处理复杂的 Shell 指令和 Git 操作，还支持多模态交互。同时，它也是一个高度可扩展的平台，积极拥抱了模型上下文协议（MCP）、A2A等拓展协议。

本文的目标将不止于工具使用，我们将深入 gemini-cli 的源码，重点分析其整体架构，并解析其命令分发、工具调用和上下文管理等关键模块的实现原理，分析其AI Agent 的整体构建思路。

我们希望这次源码层面的分析，能为那些对 AI Agent 内部机制感到好奇，或是希望构建类似终端agent的开发者，提供一份务实且有价值的技术参考。

二、核心能力演示

在深入源码之前，让我们通过几个真实的场景，直观地感受 gemini-cli 如何将自然语言转化为实实在在的生产力。

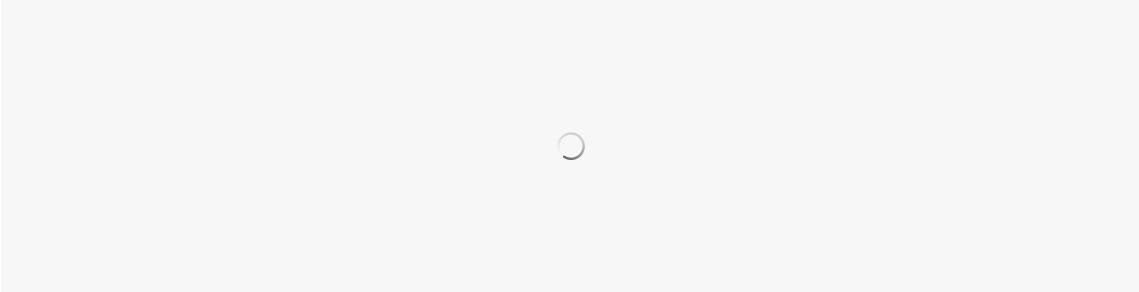
2.1 场景一：自动化系统任务 —— 批量处理文件

这是最能体现命令行工具本质的场景。gemini-cli 可以将你用自然语言描述的复杂文件操作，直接翻译成可执行的命令。

上下文： 你的一个目录下堆满了jpg图片文件。

用户输入： gemini > 将这个目录里的所有图片转换为 png 格式，并根据照片的 EXIF 数据里的拍摄日期来重命名它们。

gemini-cli的决策与执行路径：展示了 gemini-cli，是一个能够规划、编码、诊断、并从失败中学习和迭代的 AI Agent。



1.规划与编码：将任务拆解为四步计划，读取图片，转换格式，重命名新文件，删除原始文件。它没有直接生成复杂的 Shell 指令，判断出运用Python 脚本来解决问题。调用WriteFile，在**本地创建了一个 convert_images.py 脚本**。

2.环境准备与自我纠错：在执行前，它预判到需要 Pillow 库，并尝试用 pip 安装。命令失败后，它能理解错误并**自动切换到 pip3** 重试，展现了环境适应能力

3.诊断与迭代：首次运行脚本时，因图片缺少 EXIF 数据而失败。gemini-cli 捕获并理解了这个错误。它没有止步于此，而是**启动了代码修复流程**：它用 **ReadFile** 读取了自己刚写的代码，然后 **WriteFile** 进行了**代码重构**，增加了一个“使用文件修改时间作为备用方案”的逻辑。

4.最终执行与验证：最后，它**运行了更新后的脚本**，成功**完成了图片格式转换与重命名**。并通过 **ReadFolder** 命令验证了最终结果，形成了一个的闭环。

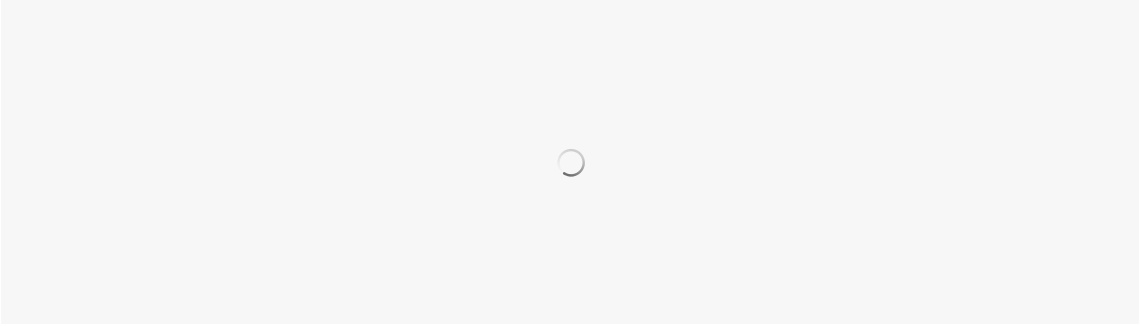
2.2 场景二：快速理解新项目 —— 深入分析代码库

面对一个陌生的代码库，它可以迅速帮助我们理清脉络。

上下文： 克隆了 gemini-cli 源代码至本地文件夹中

用户输入： gemini > 本文件中有gemini-cli的源码，帮我梳理gemini-cli的整体架构。关键的目录和它们的作用分别是什么？

gemini-cli的决策与执行路径：这个交互展示了 gemini-cli 作为信息分析助手的智能工作流。



1.环境了解与信息收集： 由于是全新会话，它首先意识到自己缺乏对当前环境的了解。它的第一步是**调用 ReadFolder 工具，扫描当前目录**，获取一份完整的文件和文件夹“地图”。

2.模式识别与策略制定： 在获取文件列表后，它利用内置的软件工程知识进行模式识别。当看到 docs/architecture.md 这个路径时，它迅速将用户的关键词“架构”与之关联，并制定出一条最高效的策略：**优先读取官方文档**，而非盲目分析代码。

3.执行与信息提取： 策略确定后，它立刻调用 ReadFile 工具，读取 docs/architecture.md 的内容。

4.整合与结构化呈现： 它没有直接返回文档原文，而是完成了**信息整合**。它将从文档中提取的信息与第一步查看的实际目录结构进行交叉验证和补充，最终生成了一份逻辑清晰、结构化的项目架构分析。

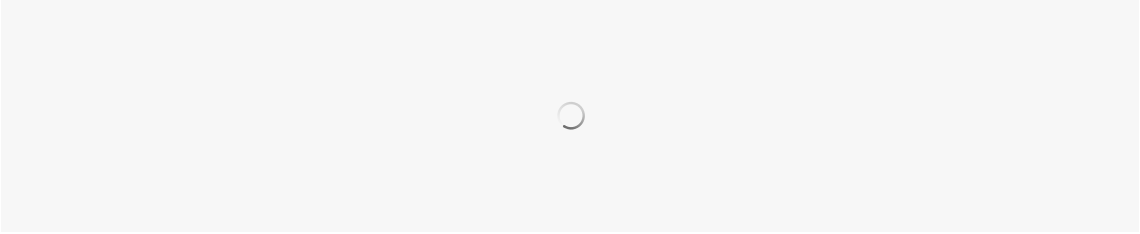
2.3 场景三：跨越模态的创造力 —— 从设计图到代码

这个场景将突破传统命令行的文本限制，理解视觉信息并将其转化为可执行的工作流。

上下文： 你的项目文件夹里有一张产品经理给的登录页面截图 login-mockup.png。

用户输入： `gemini > 这是我们的登录页面设计图 (login-mockup.png)，帮我生成对应的 HTML 和 CSS 代码，并用浏览器打开预览编码结果。`

gemini-cli的决策与执行路径： 这是从视觉到代码，再到预览的自动化工作流。



1.任务拆解与规划： 它首先将用户的复合指令**拆解为三步计划：分析图片、生成文件、打开浏览器预览**，展现了清晰的逻辑规划能力。

2.视觉理解与代码翻译： 它通过 **ReadFile 加载图片**，其多模态模型解析出设计图中的布局、组件和样式，然后基于这些视觉信息**编写结构化的 HTML 和 CSS 代码**。

3.遵循最佳实践的文件操作： 它将结构和样式分离，通过两次WriteFile调用，分别**创建了 index.html 和 style.css 两个文件**，遵循了前端开发的基本规范。

4.无缝的工具链调用： 最后，它**调用 Shell 工具执行 open 命令，自动在浏览器中打开了生成的页面**，完成了从一个创意原型到可交互预览的完整闭环。

通过这三个场景，我们看到 gemini-cli 扮演了三个关键角色：能够自我纠错的 AI Agent、高效的代码分析师，以及理解视觉的创意开发者。它的能力已远超传统命令行终端的范畴，不只是简单的命令执行者，而是能将用户的复杂意图无缝转化为实际工作流的智能伙伴。

那么，这一切强大能力的背后，究竟是怎样的架构设计在支撑呢？



腾讯技术工程

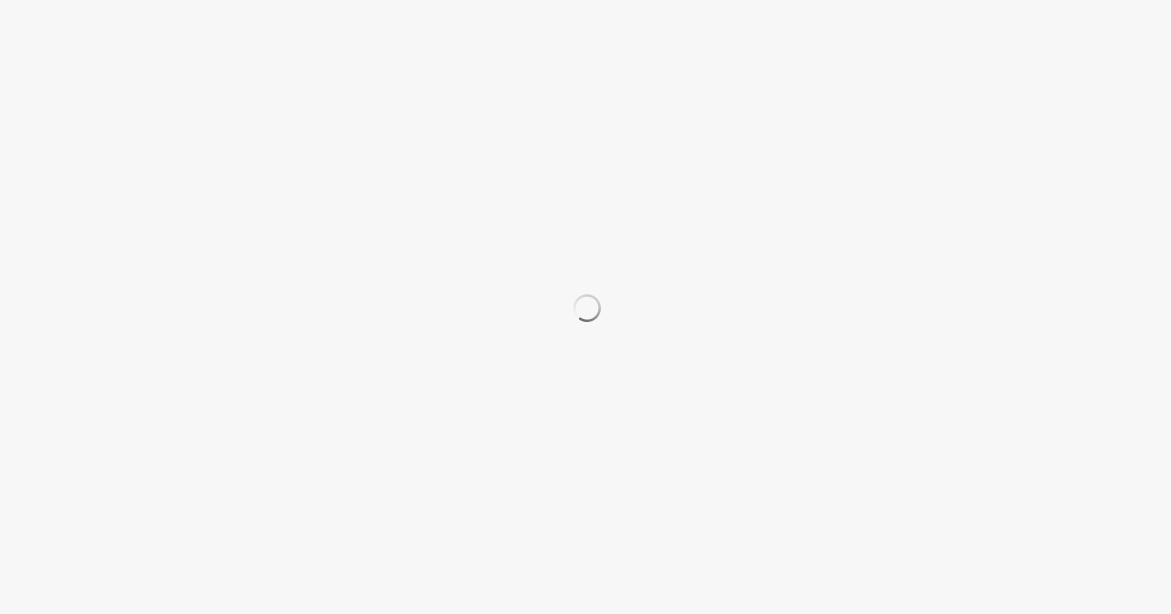
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

三、架构设计与核心源码解析

3.1 核心架构与工作流程



如图所示，gemini-cli的架构分为用户交互层、逻辑层、大模型服务与工具层。整体上围绕一个动态的推理与行动（ReAct）工作流构建。其实现逻辑分布在两个核心包中：packages/core 负责提供与大模型通信、执行工具等原子能力，而 packages/cli 作为上层协调者，负责驱动整个任务流程。

具体来说，整个ReAct循环由cli包驱动。当接收到用户指令后，它首先调用 core 包进行“推理”，core 则将模型制定的行动计划返回给 cli。收到计划后，cli 再指令 core 执行工具，并通过回调函数回收执行结果。cli 随即发起携带新上下文的下一轮“推理”，如此循环往复，直至任务完成。

这种由 cli 驱动、core 执行的协作模式，既保证了核心 AI 逻辑的可复用性，也赋予了界面层精细控制任务每一步状态的能力。

3.2 推理与行动循环

如图，Gemini CLI 的核心是一个由 packages/cli 驱动、packages/core执行的“推理-行动”（ReAct）循环。这个循环并非简单的函数递归，而是通过一系列精心设计的 Hooks和回调函数闭合的异步流程。让我们从源码的视角，一步步拆解这个循环的实现机制。

3.2.1 “用户输入”到“模型推理”

在交互模式下，每一次用户提交输入，都会触发 packages/cli/src/ui/hooks/useGeminiStream.ts 中的 submitQuery 函数。这是整个 ReAct 循环的入口。

1.预处理与分发：submitQuery 首先会调用 prepareQueryForGemini 函数。这是一个关键的预处理器，它负责解析用户输入，处理 @文件引用和 /命令等前端指令。如果指令可以在本地处理（如 /clear），流程就此终止；否则，它会将处理后的查询内容打包，准备发往模型。

2. 发起推理：拿到处理好的查询后，submitQuery 会调用 geminiClient.sendMessageStream()，与 Gemini大模型建立一个流式连接。这标志着“推理”阶段的开始。

3.2.2 “行动计划”到“工具执行”

模型在“推理”后，可能会返回一个包含行动计划的响应，即工具调用请求（ToolCallRequest）。

1.接收与调度：useGeminiStream 中的 processGeminiStreamEvents 函数负责监听事件流。当它收到 ToolCallRequest事件时，会将这些请求收集起来，并调用 scheduleToolCalls 函数。

2.UI 层的调度桥梁：scheduleToolCalls 来自 packages/cli/src/ui/hooks/useReactToolScheduler.ts。这个 Hook不直接执行工具，它实例化了来自 core 包的 CoreToolScheduler 类，并将工具请求转发给它。

3.Core 层的执行核心：真正的工具调度和执行逻辑位于 packages/core/src/core/coreToolScheduler.ts 的 CoreToolScheduler 类中。它的 schedule方法执行的关键步骤：

排队：确保同一时间只处理一批工具调用。

验证：检查工具是否存在、参数是否合法。

确认：对于敏感操作（如写文件），会挂起并等待用户在 UI 层面确认。

执行：调用工具自身的 execute() 方法，真正执行文件读写、命令运行等操作。

到此，cli 包成功地将“行动”指令委托给了 core 包去执行，并可以执行本地工具

3.2.3 “观察”结果到再次“推理”

工具执行完毕后，结果需要被送回模型，以完成“观察”并开启下一轮“推理”。这个递归流程是通过回调链实现的。

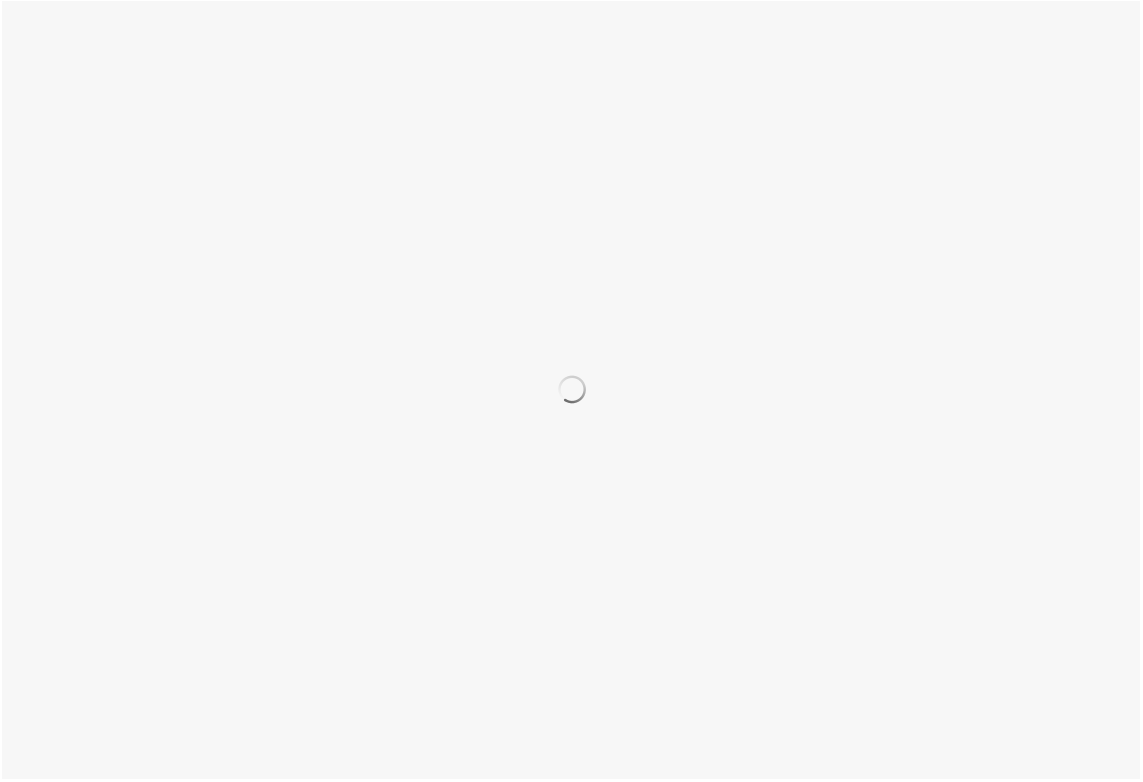
1. 执行完成：CoreToolScheduler 在所有工具执行完毕后，会调用一个名为 onAllToolCallsComplete 的回调函数，并将所有工具的执行结果作为参数。

2.逐层返回：这个回调函数会穿透 useReactToolScheduler，最终在 useGeminiStream hook 中被一个名为 handleCompletedTools 的函数接收。

3.闭合循环：handleCompletedTools 函数是整个循环的闭合点。它将所有工具的返回结果打包成模型需要的格式，然后——也是最关键的一步——**再次调用 `submitQuery` 函数**，并将工具结果作为新的查询内容，也是新一轮“观察”阶段的结束和新一轮“推理”的开始模型会根据工具的执行结果，决定是给出最终答案，还是规划下一步新的“行动” 。这个由回调驱动的循环会持续进行，直至最终任务完成。

3.3 工具调用与扩展

上一节我们看到，ReAct 循环的其中一个关键点在于模型能够决定何时调用工具。本节将深入探讨工具调用，以及 gemini-cli 是如何实现并安全执行它们的。整体逻辑可参考下图：



3.3.1 工具的声明与实现

以 read_file 工具为例（位于packages/core/src/tools/read-file.ts）。

声明：工具首先需要向大模型进行自我介绍。这是通过一个符合 JSON Schema 规范的配置对象完成的。它详细定义了工具的名称（如read_file）、功能描述（用于读取文件内容），以及最重要的——参数（如 absolute_path、offset等）。这份介绍是模型理解并决定如何使用该工具的依据。

实现：当模型决定调用该工具后，其实际的执行逻辑则被封装在一个 Invocation 类中。该类的execute()方法包含了真正的功能代码，例如调用文件系统 API 来读取文件。

3.3.2 统一的“工具注册表”

所有内置或外部的工具，会被工具注册表ToolRegistry（位于 packages/core/src/tools/tool-registry.ts）来管理。

在应用启动时，Config模块会负责创建所有内置工具的实例，并逐一注册到表中。当需要与大模型交互时，系统会从注册表中提取所有工具的“声明”信息，统一提交给模型，让模型知道它当前拥有哪些可用的能力。同样，当模型请求执行某个工具时，调度器也会通过注册表来查找并执行它。

3.3.3 安全执行的确认机制

对于 run_shell_command （实现位于packages/core/src/tools/shell.ts）这类可能造成风险的执行工具，gemini-cli 建立了一套安全确认机制，其决策逻辑位于 core 包，而 UI 交互则由 cli

包呈现。

1.决策点： 当一个高危工具被请求执行时，CoreToolScheduler 会首先调用该工具的 shouldConfirmExecute() 方法，以确定工具能否被执行。

2.检查: 以shell工具为例，shouldConfirmExecute() 会进行检查：该命令是否在用户本次会话中已经授权过?

3.请求确认: 如果授权检查未通过， CoreToolScheduler 会暂停执行，并发送信号至cli层。cli包接收到这个信号后，会渲染一个对话框，展示待执行的命令，并让用户做出是否确认执行的决定。

这套机制确保敏感操作都在用户的明确授权下进行，同时通过会话内白名单等方式避免了不必要的重复确认。

3.3.4 MCP拓展：连接外部工具

gemini-cli通过支持“模型上下文协议”（MCP），可以与任何外部工具服务器进行交互，从而获得扩展能力。

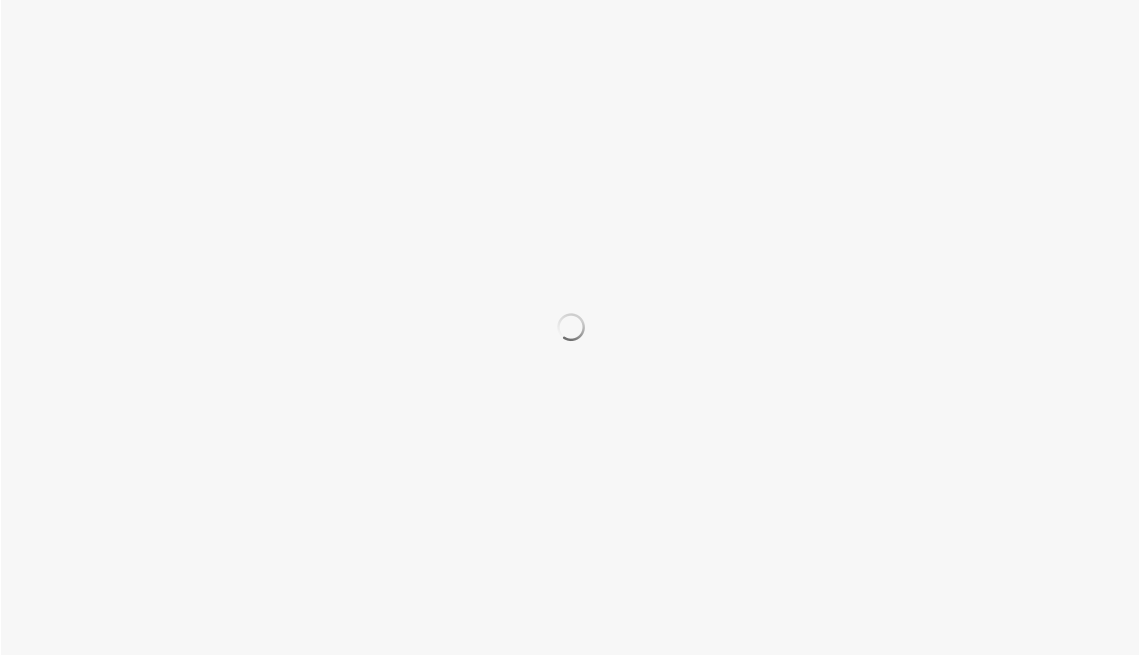
gemini-cli启动时，gemini-cli 会连接在配置中指定的 MCP服务器，并请求对方提供其支持的工具列表。MCP服务器返回工具的声明信息后，gemini-cli 会将它们作为MCP工具注册到自己的ToolRegistry 中。（代码位于packages/core/src/tools/ 目录下的 mcp-client-manager.ts）

当模型决定调用一个MCP外部工具时，其执行流程与内置工具不同是 execute()方法，它通过初始化阶段建立的连接，将参数转发给MCP服务器。它会等待服务器返回执行结果，再将其递交给模型。（代码位于packages/core/src/tools/ 目录下的 mcp-client.ts 、 mcp-tool.ts ）

通过这种方式，gemini-cli 将MCP工具整合进了自身的 ReAct循环中。

3.4 上下文管理机制

在前两节中，我们分析了 gemini-cli 的推理循环与工具执行能力。要让这些能力发挥最大价值，其管理和利用上下文的能力必不可少。gemini-cli 通过多层次上下文管理机制，实现了短期会话记忆、长期知识注入，确保了交互的连贯性。



3.4.1 会话历史

基础的会话历史，保证了模型能记住你和它之间的对话。

1.UI 层：useGeminiStream hook **(位于 `packages/cli/src/ui/hooks/useGeminiStream.ts`)**负责管理一个用于界面渲染的历史记录数组。当用户或模型产生新消息时，它会更新这个数组，让用户能在屏幕上看到对话流。

2.core层: 模型记忆维护在调用Gemini的客户端代码 GeminiClient **(位于 `packages/core/src/core/client.ts`)** 及其内部的 GeminiChat 对象 **(位于 `packages/core/src/core/geminiChat.ts`)** 中。GeminiChat 内部维护着一个需要发送给 API 的、完整的对话历史。当它处理一个新的用户请求时，其sendMessageStream 方法会执行对历史会话的“读-写”操作：

1.写入用户输入: 在向模型发送请求**前**，它首先将用户的最新输入追加到其内部历史记录的末尾。

2.发送完整历史: 它将这个包含了最新输入的完整历史发送给模型。

3.写入模型回复: 在接收到模型的完整回复**后**，它再将模型的回复追加到历史记录中。

通过这种方式，利用GeminiChat 这个有状态的对象，每一轮对话都建立在完整的历史上下文之上，实现了连贯的交互。

3.4.2 系统提示词

在大模型推理中，我们需要模型在整个会话中都记住一些核心背景信息，gemini cli除了内置的系统提示词（system prompt），还支持可选的用户自定义的上下文信息。

用户自定义的上下文，可通过 context.fileName 属性（配置文件.gemini/config.yaml）来配置，可将用户提供的文件内容，与gemini-cli内置的系统提示词（system prompt）合并，形成一个在整个会话中保持不变的预设指令。在 gemini-cli 启动时，loadCliConfig 函数 **(位于 `packages/cli/src/config/config.ts`)** 会读取 context.fileName 指定的文件内容，并将其作为 userMemory 属性，存入 Config 对象中。

在创建聊天会话时，GeminiClient 会调用 getCoreSystemPrompt 函数 **(位于 packages/core/src/core/prompts.ts)**来构建完整的系统提示词（system prompt）。这个函数执行了如下逻辑：

1.加载基础系统提示词（system prompt）：它首先会加载一个内置的字符串 basePrompt。这个 basePrompt 就是 gemini-cli 的原生系统提示，它详细定义了AI 助手的核心职责、工作流程、行为准则等。

2.追加用户上下文: 它会检查 userMemory（即您文件中的内容）是否存在。如果存在，它会将其用分隔符（---）追加到 basePrompt 的末尾。

这个合并了原生指令和用户自定义知识的、完整的系统提示词（system prompt），会在API 请求上下文中被发送。模型会基于这个上下文来进行推理和行动。

3.4.3 即时引入上下文

为了让用户能灵活地、即时地引入文件作为上下文，gemini-cli 提供了 @ 引用功能。这套机制是一个在请求发送前的预处理器中完成。

1.输入检测、解析: 用户输入会被useGeminiStream hook (位于 packages/cli/src/ui/hooks/useGeminiStream.ts)检测输入中的 @ 符号，并将其交给 handleAtCommand 处理器（位于 packages/cli/src/ui/hooks/atCommandProcessor.ts）。

2.复用存量工具: handleAtCommand 会解析出所有文件路径，然后调用上文提到的内置的文件读取工具，来读取这些文件的内容。

3.组装上下文: 它将读取到的文件内容，连同用户的原始提问，一起组装成一个完整 prompt。

4.模型推理: 最后，这个包含了“即时上下文”的查询被发送给模型。这使得模型可以在当前这一轮对话中，精准地针对特定文件内容进行回答。

3.4.4 会话持久化

默认情况下，对话历史是存在于内存中的，关闭即消失。为了能跨越多次启动来保留对话，gemini-cli 提供了快照功能。

该功能通过 /chat save和 /chat **resume**命令来触发。

具体实现是通过Logger类（位于 packages/core/src/core/logger.ts）提供了 saveCheckpoint 和 loadCheckpoint 方法。通过读写本地文件实现上下文存储和读取。

通过这四种机制的协同工作，使得gemini-cli能处理流畅的即时对话，加载长期的背景知识、引用临时的文件内容，并能按需持久化和恢复整个对话，为用户提供了强大而灵活的上下文管理能力。

四、架构思想与新范式

上一节，我们深入源码的细节，揭示了 gemini-cli 的工作原理，同时也能逐步窥见代码背后一系列深思熟虑的架构决策。本章节将尝试对具体的技术实现进行梳理和提炼，分享对gemini-

cli架构思想和技术选型的一些思考。

我们将主要探讨以下几个方面：

- gemini-cli 是如何通过分层设计，构建其可移植的 Agent 内核的？
- 大模型在其运行时扮演了怎样的动态调度角色？
- 系统是如何通过指令确认机制，来构建人机协作模式的？
- 它又是如何通过开放协议，来构想其未来的工具生态的？

通过对这些问题的探讨，希望能够揭示其设计背后的决策考量，为构建同类 AI Agent 的开发者提供一份有价值的架构参考。

4.1 可复用的Agent 内核架构

gemini-cli 在架构上对项目模块进行了明确的职责划分，将**核心AI逻辑**与**前端交互界面**彻底分离，其目的是构建一个与特定运行时环境解耦、可独立复用的Agent内核。

这一模式体现在 packages 目录下两个关键模块的划分上：

- **packages/core (内核层)**: 此模块封装了所有与 AI Agent 相关的原子能力和核心逻辑。它包含了与 Gemini API 的通信客户端 (GeminiChat)、工具的定义与注册机制 (ToolRegistry)、工具调度的执行器 (CoreToolScheduler)，以及会话日志与快照功能 (Logger)。内核层不包含任何与用户界面 (UI) 相关的代码，其 API 设计与具体应用无关的。
- **packages/cli (应用交互层)**: 此模块是 core 内核的一个具体应用，负责将其能力适配到命令行终端这个特定的运行环境中。它的职责包括：解析命令行参数、使用 Ink 库渲染终端 UI、管理用户交互状态 (Node.js，基于Hooks和回调的事件驱动非阻塞 I/O 模型)，以及调用 core 模块提供的 API 来驱动整个 ReAct 工作流。

在我们前文的源码分析中，两个模块的职责边界很清晰：

- **ReAct 循环的实现**：主逻辑 useGeminiStream Hook 存在于 cli 交互层，而调用的 sendMessageStream 等核心通信逻辑是由 core 内核层提供。
- **安全确认机制**：需要用户确认的决策由 core 内核层的 CoreToolScheduler 做出，但应用渲染确认框的 UI 逻辑则由 cli交互层实现。
- **上下文处理**：cli 交互层负责解析@ 符号这类特定于终端的便捷语法，并将解析出的文件内容，传递给 core 来构建最终的请求上下文。

可复用的Agent 内核带来了较强的可移植性，作为一个独立的模块，可以被应用在任何项目中，如构建 web 服务、桌面应用等，它是可供开发者在不同场景下构建AI Agent的核心 SDK。

4.2 LLM作为动态调度器的开发范式

在传统应用中，程序控制流由开发者通过 if/else 等逻辑硬编码。而这里gemini-cli展示了大模型时代的编程范式，它将大语言模型作为应用的计划器、调度器。

开发者负责通过 ToolRegistry 声明式地注册一系列原子化的工具，但不编写具体的业务流程代码，而如何组合这些工具的决策权，交给了 LLM。当用户输入复杂指令时，LLM 在运行时动态生成一个执行计划，由具体代码（coreToolScheduler.ts）完成执行。

这种模式带来了两大优势：

- **能力的涌现**：由于执行计划是LLM动态生成的，agent能够执行开发者从未明确编码过的行为，甚至自主决定编写并执行一个脚本来完成任务。
- **业务逻辑复杂度降低：**开发者只需不断增加原子能力工具，复杂的业务编排逻辑交给 LLM 来决定，整体代码复杂度得以降低。

gemini-cli 将 LLM 作为动态调度器的设计，是当前 AI Agent 领域的核心实现范式。无论是 LangChain、LlamaIndex 等框架，还是 OpenAI、Grok 等模型提供商的官方工具调用功能，其底层逻辑都是一致的，即，将LLM做为一个主动的任务规划与函数调用引擎。

此架构范式让应用的能力上限不再受限于开发者预设的控制流，而是取决于 AI 在运行时对可用工具的动态组合与调用，为实现能处理复杂、多步任务，并具备一定自主性的通用 AI agent 提供了可参考的实现路径。

4.3 指令确认与安全执行的人机共创模式

当 LLM 成为动态调度器，具备自主规划和执行任务的能力时，问题也随之而来：如何确保AI的行为可控?

被业界广泛认可的解决方法是利用**Human-in-the-Loop**设计模式来构建自动化流程。HITL 强调在 AI 系统的关键决策点上，引入人类的监督、审核或最终授权，完成流程闭环。

gemini-cli 提供了将HITL理念工程化的范例。实现方案上并不是直接限制 AI模型本身的能力，而是通过指令确认与安全执行机制，在终端环境中实现了一种人机共创的协作模式。即，AI 负责提议与规划，人类保留最终的决策与执行权。

从源码层面看，为了保留人的决策权，CoreToolScheduler 在执行 run_shell_command 等高风险工具前，会暂停 ReAct 循环并向 cli 应用层发起授权请求。代码设计上做了职责分离：core 内核负责决策，cli层负责交互，利用了nodejs的事件驱动模型，实现了非阻塞式的确认流程。

另外，在上下文管理机制中，gemini-cli 允许我们使用 @file 实时引用文件，可以为 AI 注入明确的上下文信息，为人保留了更多的上下文控制权限、个人知识库信息的主导权。

可以说，为了实现HITL的设计理念，gemini-cli的采用的工程化手段，给了开发者构建人机协同的 AI Agent 提供了有价值的设计参考。

4.4 从工具到agent协作的开放协议生态

gemini-cli 在扩展性上拥抱了开放协议MCP、A2A，在拓展性上不限于官方与开发者自建的插件工具。

- **MCP，实现能力即插即用：通过MCP协议将**工具解耦为可通过网络调用的独立服务。在 gemini-cli 的实现中，McpClientManager 在启动时会向配置的MCP服务器查询并加载工具的元信息，将其注册进本地的 ToolRegistry。当 LLM 决定调用时，将请求**转发**给对应 MCP 服务， 为gemini-cli 提供各类第三方能力集成的可能。
- **A2A，将gemini-cli的能力服务化以支持智能体间的协作：**近期gemini-cli开源了对A2A的支持，在 packages/a2a-server 中的实现。它将 gemini-cli 的核心能力封装成了一套可通过A2A协议调用的服务。尽管A2A整体生态尚处于早期阶段，但该实现对未来gemini-cli成为分布式Agent协作网络中的一部分，提供了可能性。

gemini-cli 通过支持 MCP 和 A2A 这两大开放协议，展示了一种双向的扩展模式：既能作为客户端使用外部工具服务，也能作为服务端为其他应用提供 Agent 能力。开发者可以用任意语言构建工具或应用与 gemini-cli 无缝集成，共同组成一个更加开放和强大的 AI Agent 生态。

五、总结与展望

从直观的能力演示到深入的源码剖析，我们一同拆解了 gemini-cli 的内部构造，并探讨了其背后的架构思想。至此，我们可以清晰地看到，gemini-cli 不止是一个功能丰富的命令行工具，它更是一个关于如何构建AI Agent 、有价值且可供参考的工程范例。

回顾全文，其核心设计亮点，可总结为如下四点：

- **代码架构：**通过“核心层-交互层”的分离，提供了一个可移植的内核sdk，提供了易于复用与拓展的工程范例。
- **运行模式：**基于ReAct框架，将 LLM 作为大脑，实现了agent能力的“涌现”。
- **人机协作：**遵循了HITL设计理念，在 AI 的自主性与人的主导权间取得平衡，构建可信的共创模式。
- **拓展生态：**通过拥抱开放协议，支持工具能力的拓展，提供多agent间协作的无限潜力。

展望未来，以gemini-cli为代表的终端Agent 可能往几个方向的发展：

- **与操作系统、桌面应用的深度集成联动：**终端Agent可能打破终端本身的限制，通过与操作系统 API、被安装的各类应用的深度集成，理解屏幕交互信息，模拟人的操作，驱动任意桌面应用，执行跨系统应用工作流。
- **更强大的长期记忆：**随着模型能力持续进化，结合向量检索等技术，Agent 将建立更大规模的长期记忆能力，能基于对整个项目的全局上下文理解，做出更优的决策与执行。
- **从单一智能体到分布式多智能体协作**：随着A2A 等协议的成熟，更加繁荣的 Agent 生态将会出现。一个复杂的开发任务可能会被分发给多个专用 Agent 协作完成，发展为从命令执行、代码开发逐步拓展到自动化测试、CI/CD流程、安全扫描等工业化生产要素完备的自动化研发Agents团队。

最终，开发者终端agent将不再孤立，而是一个由 AI Agents驱动的、高度协同的智能开发环境。开发者的角色也将从代码开发、执行者，转变为与AI agents协同共创的架构师。

六、参考文献

1. gemini-cli 官方 GitHub 仓库:**https://github.com/google-gemini/gemini-cli**
2. Gemini API 函数调用官方文档: **https://ai.google.dev/gemini-api/docs/function-calling?hl=zh-cn&example=meeting**
3. [论文]Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *ArXiv, abs/2210.03629*.**https://arxiv.org/abs/2210.03629**
4. [论文]Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Welleck, S., Majumder, B., Gupta, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *ArXiv, abs/2303.17651*. **https://arxiv.org/abs/2303.17651**
5. [论文]Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *ArXiv, abs/2302.04761*.**https://arxiv.org/abs/2302.04761**
6. 模型上下文协议 (MCP) 规范：**https://github.com/modelcontextprotocol**
7. Agent-to-Agent (A2A) 通信协议：**https://github.com/a2aproject/A2A**
8. 人机回路 (Human-in-the-Loop, HITL) 概念：**https://en.wikipedia.org/wiki/Human-in-the-loop**
9. 六边形架构 (端口与适配器): **https://alistair.cockburn.us/hexagonal-architecture/** https://alistair.cockburn.us/hexagonal-architecture



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容

公众号



阅读 1.4万

留言 6

写留言

EasonFeng5… 北京 2025年11月24日 4
拆解的再深入一些会更好

孙振强+AI进… 陕西 2025年11月24日 1
从自然语言指令到自动化工作流，Google gemini-cli展示了终端AI Agent的完整实现。无论是批量处理文件还是解析代码库，其自我纠错与规划能力让人印象深刻。

瑾言 上海 2025年11月24日 1
不错👍

风筝 四川 2025年11月24日 1
虽然看不懂，但是感觉很厉害

Ty 山东 2025年11月24日 1
好文章

陈浩能 广东 2025年11月24日
难得好文